

# Q-Kernel: Exact Odd- $Q$ Contextuality Kernel Extraction for Weyl/Pauli Measurement Programs

Manuel Flores Gordillo

July 5, 2026

## Abstract

Q-Kernel is a research-software implementation of an exact odd- $Q$  contextuality criterion for finite Weyl/Pauli measurement programs. Given an explicitly specified context family, the library builds a context-observable incidence matrix over  $\mathbb{F}_2$ , computes integer commutator-carry bits, searches for a closed cycle with odd carry, extracts a smaller contextuality kernel, and exports independently verifiable certificates. The implementation is deliberately conservative: it is a contextuality-kernel extractor, not a magic-state optimizer or a hardware-resource estimator. The package provides multiple exact solvers, connected-component decomposition, metadata-aware observable identity semantics, SAT/MaxSAT export, external-solver output import, stable certificate verification, a one-call obstruction-spectrum lookup, and export of a runnable *sequential-measurement* hardware protocol (each context observable measured separately, so the statistic is genuinely device-dependent rather than algebraically fixed).

## 1 Purpose and scope

The goal of Q-Kernel is to turn an exact finite contextuality criterion into a reproducible compiler-analysis primitive. The novelty target is the odd- $Q$  carry/kernel/certificate implementation, not the GF(2) linear-system framing itself. The input is not a raw list of Pauli operators, but a measurement program: observables together with the contexts in which they are jointly measured or algebraically asserted.

The safe claim is:

Q-Kernel extracts exact odd- $Q$  contextuality kernels.

The unsafe claim is:

Q-Kernel is a magic-state,  $T$ -count, or distillation optimizer.

The latter would require an additional bridge theorem or robust empirical evidence. This paper therefore presents Q-Kernel as a certificate-producing contextuality analyzer.

## 2 Related work and novelty hygiene

The binary multi-qubit case has an established finite-geometric formulation in terms of symplectic polar spaces over  $\mathbb{F}_2$ . In particular, de Boutray, Holweck, Giorgetti, Masson, and Saniga formulate a broad class of observable-based contextuality proofs as the absence of solutions to a linear system and study contextuality degree for subgeometries of binary symplectic polar spaces [?]. Subsequent

work by Muller, Saniga, Giorgetti, de Boutray, and Holweck develops algorithms and bounds for contextuality degree in multi-qubit symplectic polar spaces [?].

Q-Kernel therefore does not claim novelty for the linear-system framing, the use of  $\mathbb{F}_2$  incidence constraints, or the binary symplectic polar-space viewpoint. Its contribution is narrower: an exact software implementation of the odd- $Q$  carry criterion with integer-lift arithmetic, a closed symplectic  $Q$ -form, qudit-aware scope beyond purely binary polar-space inputs, minimum-kernel extraction, explicit observable/event identity semantics, external SAT/MaxSAT export/import, and standalone verifiable certificates.

### 3 Weyl/Pauli measurement programs

Fix local dimension  $d$  and  $m$  sites. Q-Kernel represents Weyl labels as vectors

$$v \in \mathbb{Z}_d^{2m}$$

with interleaved coordinates

$$[z_1, x_1, z_2, x_2, \dots, z_m, x_m].$$

The integer symplectic pairing is

$$\langle v, w \rangle = \sum_{k=1}^m (x_k z'_k - z_k x'_k),$$

where the right-hand side is evaluated on integer lifts.

A context  $C = (c_1, \dots, c_r)$  is valid when the labels commute pairwise and sum to zero modulo  $d$ . The commutator carry bit is

$$b(C) = \sum_{i < j} \frac{\langle c_i, c_j \rangle}{d} \pmod{2}.$$

The division by  $d$  is performed before reducing modulo 2. Reducing the symplectic pairing modulo  $d$  first destroys the carry information and gives the wrong invariant.

**Proposition 1** (Integer carry rule). *For a valid context  $C$ , pairwise commutation implies  $\langle c_i, c_j \rangle \equiv 0 \pmod{d}$ , so each quotient  $\langle c_i, c_j \rangle/d$  is defined on integer lifts. The carry bit  $b(C)$  is the parity of the sum of these integer quotients. It is not recoverable from the symplectic pairings after reducing them modulo  $d$ .*

In the implementation this rule is enforced by `qkernel.symplectic.symplectic_int` and `qkernel.carry.compute_b`.

### 4 Incidence cycles and the odd- $Q$ criterion

Given contexts  $C_1, \dots, C_n$  and observable names  $o_1, \dots, o_s$ , Q-Kernel builds the incidence matrix

$$A \in \mathbb{F}_2^{n \times s}$$

where  $A_{ij} = 1$  iff observable  $o_j$  appears in context  $C_i$ .

A binary vector  $\lambda \in \mathbb{F}_2^n$  selects a collection of contexts. It is closed when

$$A^T \lambda = 0.$$

For even  $d$ , the implemented criterion is

$$\text{contextual} \iff \exists \lambda : A^T \lambda = 0, \quad b^T \lambda = 1.$$

**Proposition 2** (Implemented odd- $Q$  criterion). *For the even- $d$  Weyl context families covered by the underlying criterion note, contextuality is witnessed exactly by a binary vector  $\lambda$  satisfying  $A^T \lambda = 0$  and  $b^T \lambda = 1$ . Q-Kernel implements this condition as its certified contextuality test.*

The quantity  $b^T \lambda$  is the odd- $Q$  obstruction.

## 5 Closed symplectic form for $Q$

**Proposition 3** (Closed symplectic form for  $Q$ ). *For a closed cycle  $\lambda$ , flatten the selected contexts into an ordered observable multiset  $v_1, \dots, v_N$ . Then*

$$Q(\lambda) = \frac{1}{d} \sum_{a < b} \langle v_a, v_b \rangle \pmod{2}.$$

*The division by  $d$  is applied to the aggregate integer numerator. Individual inter-context pairings need not be divisible by  $d$ .*

*Proof.* The closed form agrees with  $b^T \lambda$ . Each selected context sum can be written in an integer lift as  $S_r = dt_r$ . Inter-context terms contribute

$$\frac{1}{d} \langle S_r, S_s \rangle = d \langle t_r, t_s \rangle,$$

which vanishes modulo 2 for even  $d$ . Thus the aggregate symplectic numerator reduces modulo 2 to the sum of intra-context carry terms.  $\square$

The implementation exposes both computations in `qkernel.closed_form` and cross-checks them on cycle-space elements.

## 6 Kernelization problem

Q-Kernel solves the minimum-context odd- $Q$  kernel problem:

$$\begin{aligned} \min_{\lambda} \quad & |\lambda| \\ \text{s.t.} \quad & A^T \lambda = 0, \\ & b^T \lambda = 1, \\ & \lambda \in \{0, 1\}^n. \end{aligned}$$

The extracted kernel is the selected subprogram induced by  $\lambda$ . Q-Kernel also reports the selected observable set and compression ratios, but those ratios are diagnostic; they are not resource monotones.

## 7 Observable identity semantics

A central implementation issue is the distinction between algebraic equality and protocol identity. Two labels may have the same Weyl vector while representing different measurement events in a circuit or schedule.

Q-Kernel therefore attaches optional metadata to observables:

$$\text{identity\_scope} \in \{\text{observable}, \text{event}\}.$$

Safe canonicalization by Weyl vector merges observable-scoped labels only. Event-scoped labels remain distinct unless an explicit force mode is selected. This avoids silently creating parity cycles that are algebraically valid but protocol-false.

## 8 Input frontends

The package currently supports four input styles:

1. Weyl JSON programs.
2. Pauli context JSON programs.
3. Grouped Pauli schedules.
4. Row-oriented Pauli table CSV/JSON.
5. A dependency-free Stim-lite MPP subset that converts explicit Pauli-product measurement blocks into schedule layers.
6. A dependency-free Qiskit-lite JSON adapter for Pauli-layer or sparse-term exports.

The Pauli table, Stim-lite, and Qiskit-lite adapters are dependency-free contracts intended for future full Qiskit, Stim, or compiler-specific frontends. A raw set of observables is not enough; contexts or layers must be explicit.

## 9 Algorithms and solvers

Q-Kernel provides several exact pure-Python solvers:

**Cycle-span enumeration.** Enumerates the span of a basis for  $\ker A^T$ . Exact and simple, but exponential in cycle-space dimension.

**Bounded-weight search.** Searches subsets by increasing Hamming weight up to a user budget. Useful when small certificates are expected.

**Branch-and-bound.** Encodes each context as

$$\text{state}_i = \text{incidence\_row}_i \parallel b_i$$

and searches for XOR target  $0 \dots 01$ , pruning any branch whose remaining suffix span cannot reach the residual target.

The library also decomposes disconnected context–observable components. If a global odd- $Q$  cycle exists in a disconnected union, at least one connected component contains an odd- $Q$  cycle. This allows independent component search and prevents irrelevant disconnected blocks from inflating the kernel search.

**Proposition 4** (Component decomposition). *Let a context family decompose into disconnected components in the context–observable bipartite graph. If a global cycle has odd  $Q$ , then at least one component has an odd- $Q$  cycle. Consequently, the minimum-cardinality odd- $Q$  kernel of the disconnected union is the minimum among the componentwise odd- $Q$  kernels.*

*Proof.* Both incidence closure and the carry parity decompose as direct sums over disconnected components. A global closed vector  $\lambda$  is the sum of its component restrictions, and  $Q(\lambda)$  is the parity sum of the component  $Q$ -values. If the total parity is odd, at least one component parity is odd. The weight of that component restriction is no larger than the global weight. Conversely, any odd component cycle is an odd global cycle after extending by zero.  $\square$

## 10 External SAT and MaxSAT workflows

The minimum odd- $Q$  problem is naturally a Boolean optimization problem. Q-Kernel exports dependency-free solver models:

- DIMACS CNF for fixed- $k$  feasibility:

$$A^T \lambda = 0, \quad b^T \lambda = 1, \quad |\lambda| \leq k.$$

- WCNF/MaxSAT for direct minimization, with parity constraints as hard clauses and unit soft clauses  $-\lambda_i$  penalizing selected contexts.

External solvers are treated as untrusted candidate generators. Q-Kernel imports only the selected  $\lambda$  variables from solver output and independently verifies the resulting certificate. The implementation also includes an optional PySAT backend and RC2 MaxSAT wrapper behind a separate dependency extra; these wrappers use the same verified certificate path as external file-based solvers.

## 11 $\mathbb{Z}_d$ valuation verification

The odd- $Q$  condition is a parity-shadow certificate. For compression in even qudit dimension, this is not by itself sufficient to guarantee that the compressed subfamily remains a genuine AvN contextual family over  $\mathbb{Z}_d$ . Q-Kernel therefore verifies the full valuation system

$$M\phi = \gamma \pmod{d},$$

where  $M$  is the context-observable multiplicity matrix and  $\gamma$  is the context scalar vector. A family is genuinely  $\mathbb{Z}_d$ -contextual when this system has no solution. The implementation solves this congruence system by Smith normal form over the integers. Thus a compressed kernel certificate is accepted only if it is both odd- $Q$  and unsatisfiable as a  $\mathbb{Z}_d$  valuation system.

If explicit context phases are supplied, they are used as  $\gamma$ . Otherwise the backward-compatible default is  $\gamma_i = (d/2)b_i$  for even  $d$  and  $\gamma_i = 0$  for odd  $d$ . High-dimensional AvN certificates should provide explicit phases whenever the operator-product scalar is known.

## 12 Certificates

A Q-Kernel certificate records:

- a canonical SHA-256 hash of the input program;
- the selected vector  $\lambda$ ;
- selected context and observable indices;
- the carry vector  $b$ ;

- the claimed  $Q$ -value;
- software version, coordinate convention, integer-carry rule, and criterion identifier;
- the independent verifier result.

**Proposition 5** (Certificate verification soundness). *If the  $Q$ -Kernel certificate verifier accepts a certificate for a supplied program, then the certificate is bound to that program hash, its selected contexts define the claimed vector  $\lambda$ , the vector is closed under the program incidence matrix, and the recomputed odd- $Q$  value is 1.*

Verification recomputes the program hash, checks  $\lambda$  against selected contexts, verifies  $A^T \lambda = 0$ , and recomputes  $Q(\lambda) = 1$ .

### 13 Peres–Mermin test case

For the two-qubit Peres–Mermin square,  $Q$ -Kernel recovers the standard six-context odd- $Q$  kernel. The carry vector has a single odd entry in the implemented convention, and the selected cycle is

$$\lambda = (1, 1, 1, 1, 1, 1).$$

The analyzer reports

$$Q(\lambda) = 1.$$

Negative controls, including row-only subfamilies and event-scoped duplicate labels, are noncontextual unless explicit force-canonicalization is requested.

### 14 Synthetic benchmarks

The current benchmark suite is intentionally modest. It measures software behavior rather than quantum resource value. Examples include:

- Peres–Mermin plus disconnected noncontextual noise blocks;
- connected zero-carry ladders;
- solver comparison across span, bounded-weight, branch-and-bound, SAT export, and MaxSAT export workflows.

These benchmarks show that decomposition and solver choice matter, but they do not establish hardware advantage or magic-resource bounds.

### 15 Fiber-lift constructor

$Q$ -Kernel includes a validated constructor for even-base fiber lifts  $d \rightarrow 2d$ . Each lifted observable is written as  $\tilde{v} = u + dx$ , with  $x \in \{0, 1\}^{2m}$ . For even base  $d$ , the conditions that lifted contexts remain closed and pairwise commuting modulo  $2d$  are linear over  $\mathbb{F}_2$ . The constructor solves these equations, validates the resulting Weyl program, lifts phases by the default inclusion  $\gamma \mapsto 2\gamma$ , and immediately runs the  $\mathbb{Z}_{2d}$  valuation check.

This constructor is a certification utility, not an optimizer. Odd base  $d$  is left for later work because the  $d^2 \langle x_a, x_b \rangle$  contribution introduces quadratic terms in the binary lift variables.

| noise | ctx | obs  | comp | kernel ctx | decomp ms | global ms | same |
|-------|-----|------|------|------------|-----------|-----------|------|
| 0     | 6   | 9    | 1    | 6          | 0.1753    | 0.1554    | True |
| 10    | 16  | 39   | 11   | 6          | 0.3401    | 0.3777    | True |
| 50    | 56  | 159  | 51   | 6          | 1.0351    | 2.6312    | True |
| 100   | 106 | 309  | 101  | 6          | 1.9503    | 8.6265    | True |
| 500   | 506 | 1509 | 501  | 6          | 10.5471   | 242.1095  | True |

Table 1: Synthetic decomposition benchmark. A Peres–Mermin core is augmented with disconnected noncontextual checks. Times are local median wall times in milliseconds and are intended as regression diagnostics, not hardware performance claims.

| case                        | solver           | ctx | obs | contextual | kernel ctx | verified | ms     |
|-----------------------------|------------------|-----|-----|------------|------------|----------|--------|
| pm_plus_0_disconnected      | span             | 6   | 9   | True       | 6          | True     | 0.1775 |
| pm_plus_0_disconnected      | bounded_weight_6 | 6   | 9   | True       | 6          | True     | 0.1665 |
| pm_plus_0_disconnected      | branch_bound     | 6   | 9   | True       | 6          | True     | 0.169  |
| pm_plus_0_disconnected      | auto             | 6   | 9   | True       | 6          | True     | 0.1679 |
| pm_plus_100_disconnected    | span             | 106 | 309 | True       | 6          | True     | 1.7172 |
| pm_plus_100_disconnected    | bounded_weight_6 | 106 | 309 | True       | 6          | True     | 1.5984 |
| pm_plus_100_disconnected    | branch_bound     | 106 | 309 | True       | 6          | True     | 1.6463 |
| pm_plus_100_disconnected    | auto             | 106 | 309 | True       | 6          | True     | 2.0006 |
| pm_plus_10_connected_ladder | span             | 16  | 29  | True       | 6          | True     | 0.2962 |
| pm_plus_10_connected_ladder | bounded_weight_6 | 16  | 29  | True       | 6          | True     | 2.3851 |
| pm_plus_10_connected_ladder | branch_bound     | 16  | 29  | True       | 6          | True     | 0.2712 |
| pm_plus_10_connected_ladder | auto             | 16  | 29  | True       | 6          | True     | 0.3595 |

Table 2: Synthetic solver comparison on selected instances. The purpose is to compare internal exact solver behavior and catch regressions; the table does not imply quantum resource advantage.

## 16 Lift pipeline

The tower workflow is exposed as an end-to-end reproducibility pipeline:

$$\text{fiber lift} \longrightarrow \mathbb{Z}_d \text{ valuation check} \longrightarrow \text{tower-law report}.$$

The pipeline emits a structured JSON/Markdown report containing the lift bits, lifted valuation result, tower generativity bit, and explicit safe/unsafe claim boundaries. This makes the tower workflow reproducible while preserving the distinction between certification diagnostics and resource optimization.

## 17 Tower and doubling scope

For valid fiber lifts  $d \rightarrow 2d$ , Q-Kernel now implements a closed-form tower/doubling generativity diagnostic. Write each lifted observable as  $\tilde{v} = u + dx$ , with  $u$  the base residue and  $x \in \{0, 1\}$  the lift-bit vector. For each flattened selected-cycle pair set

$$M_{ab} = \langle u_a, x_b \rangle + \langle x_a, u_b \rangle.$$

The certified generativity bit is

$$G = \left\lfloor \frac{\sum_{a < b} M_{ab}}{2} \right\rfloor \oplus \left\lfloor \frac{K}{2} \right\rfloor \pmod{2},$$

where  $K = \#\{(a, b) : M_{ab} \text{ is odd}\}$ . The cycle is non-generative iff  $G = 0$ . The direct sum  $\sum_{a < b} M_{ab}$  is used; intrinsic reconstructions of this term are not used because they can differ by a mod-four carry on repeat cycles.

This certifies the tower/doubling generativity bit for the stated fiber-cycle setting. It does not certify tower compression, resource advantage, or compiler optimization.

The tower/doubling direction is deliberately experimental. Current evidence supports a repeat-free subclass where lift-induced carry has a closed bit formula. Repeat-observable corrections remain open. Q-Kernel therefore exposes only a scope reporter for tower/doubling experiments and does not certify tower compression, free resource activation, or an additive resource-gauge interpretation.

## 18 Compiler optimizer path

Q-Kernel can become part of a quantum compiler optimizer, but not by treating kernel compression itself as a circuit-resource theorem. The present software optimizes a diagnostic object: the minimum odd- $Q$  contextuality kernel. A true compiler optimizer would require an additional layer, such as a semantics-preserving rewrite system, a backend resource model, an empirical resource correlation, or a theorem linking odd- $Q$  kernels to a compiler resource.

The implementation now exposes a compiler-facing diagnostic report and a conservative before/after pass comparison. These reports explicitly state that a candidate rewrite requires a separate semantic-equivalence proof. Thus Q-Kernel can guide optimizer development without claiming that a smaller contextuality kernel is already a smaller  $T$ -count, lower magic cost, or hardware improvement. A small compiler-pass playground illustrates this distinction: removing disconnected nonkernel checks preserves the detected odd- $Q$  kernel while reducing the diagnostic input size, but the legality of that removal remains an external compiler-semantics obligation. To make this boundary explicit, Q-Kernel includes a rewrite-candidate policy registry separating safe diagnostic pruning, rewrites requiring external semantics, experimental resource probes, and forbidden resource claims.

## 19 Limitations

Q-Kernel does not currently claim:

- a magic-state monotone;
- a  $T$ -count or  $T$ -depth lower bound;
- stabilizer-rank estimates;
- hardware overhead estimates;
- certified tower/doubling compression;
- that passive dimension embedding is operationally free.

The odd- $Q$  obstruction is an exact contextuality certificate. Any resource-estimation interpretation is future work.

## 20 Release audit

The repository includes a release audit command that checks the safety-critical invariants of the artifact: odd- $Q$  and  $\mathbb{Z}_d$  verification on the Peres–Mermin example, rewrite-policy guardrails, compiler-playground diagnostic status, fiber-lift construction, tower-law reporting, lift-pipeline output, and novelty hygiene against finite-geometric prior art. The audit is not a substitute for the proofs, but it is a reproducibility and release-readiness checklist intended to prevent accidental overclaiming.

## 21 Roadmap

Immediate technical directions:

1. Expand optional PySAT/RC2 integration and benchmark it against external file-based solvers while keeping the core dependency-light.
2. Optional full Stim bridge built on top of the current Stim-lite MPP adapter.
3. Optional Qiskit bridge emitting the Pauli table format.
4. More realistic benchmark families.
5. Empirical correlation studies between kernel metrics and known circuit-resource metrics, without treating correlation as theorem.
6. Further formalization of tower/doubling laws, especially repeat-observable corrections.

## 22 Code-to-proposition map

The main theorem-level claims are intentionally tied to implementation modules:

- Proposition ?? : `symplectic.py`, `carry.py`, `test_integer_carry_bug.py`.
- Proposition ?? : `incidence.py`, `analyzer.py`, `optimizer.py`.
- Proposition ?? : `closed_form.py`, `test_closed_q_form.py`.
- Proposition ?? : `decompose.py`, `test_decompose.py`.
- Proposition ?? : `certificate.py`, `verify.py`, `solver_output.py`, `valuation.py`.